EX.NO: 1 **DATE:**	**Write a program to perform basic image processing operations such as histogram equalization, thresholding, edge detection (e.g., Canny or Sobel), morphological operations (e.g., dilation, erosion), and apply basic data augmentation techniques (e.g., rotation, flipping, scaling) on a sample image**

AIM

To write and execute a Python program to perform basic image processing operations such as histogram equalization, thresholding, edge detection using Canny, morphological operations such as erosion and dilation, and basic data augmentation techniques such as rotation, zooming, and horizontal flipping on a sample image.

ALGORITHM

1. Import the required libraries: OpenCV, NumPy, Matplotlib, and TensorFlow Keras.
2. Read the sample image in grayscale using OpenCV.
3. Perform histogram equalization using cv2.equalizeHist() and display the original and equalized images along with their histograms.
4. Apply Gaussian Blur to the grayscale image to reduce noise.
5. Perform Canny edge detection using threshold values 100 and 200, and display the detected edges.
6. Read the image in RGB format and prepare it for data augmentation.
7. Apply rotation, zooming, and horizontal flipping using ImageDataGenerator.
8. Generate and display eight augmented images along with the original image.
9. Convert the grayscale image into a binary image using a threshold value of 127.
10. Create a kernel and perform erosion and dilation on the binary image.
11. Display the binary image, eroded image, and dilated image, and perform binary thresholding on the original image.
12. Display all the processed images and observe the results of the different image processing operations.

SOFTWARE REQUIREMENTS

Software:

- Python 3.x
- OpenCV
- NumPy
- Matplotlib
- TensorFlow / Keras

Development Environment:

- Jupyter Notebook / VS Code / PyCharm
- Windows / Linux / macOS

PROGRAM

```
from matplotlib import pyplot as plt
import cv2
import numpy as np
#########################################################################
##
# Histogram Equalization
img = cv2.imread(r"C:/Users/mones/Desktop/DL/Exp_1/Image_1.jfif", 0)
equ = cv2.equalizeHist(img)
# Histograms
hist1 = cv2.calcHist([img], [0], None, [256], [0, 256])
hist2 = cv2.calcHist([equ], [0], None, [256], [0, 256])
# Display images and histograms
plt.figure(figsize=(10,8))
plt.subplot(221)
plt.imshow(img, cmap='gray')
plt.title('Original Image')
plt.axis('off')
plt.subplot(222)
plt.imshow(equ, cmap='gray')
plt.title('Equalized Image')
plt.axis('off')
plt.subplot(223)
plt.plot(hist1, color='blue')
plt.title('Original Histogram')
plt.subplot(224)
plt.plot(hist2, color='blue')
plt.title('Equalized Histogram')
plt.tight_layout()
plt.show()
```

```
#########################################################################
#######
# Edge Detection
img = cv2.imread(r"C:/Users/nagar/Desktop/DL/Exp_1/Image_1.jfif", 0)
blurred = cv2.GaussianBlur(img, (5, 5), 0)
edges = cv2.Canny(blurred, 100, 200)
plt.figure(figsize=(10,8))
plt.subplot(121)
plt.imshow(img, cmap='gray')
plt.title('Original Image')
plt.axis('off')
plt.subplot(122)
plt.imshow(edges, cmap='gray')
plt.title('Edge Detection')
plt.axis('off')
plt.show()
#########################################################################
# Data Augmentation
from tensorflow.keras.preprocessing.image import ImageDataGenerator
img = cv2.imread(r"C:/Users/nagar/Desktop/DL/Exp_1/Image_1.jfif")
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
img = np.expand_dims(img, axis=0)
datagen = ImageDataGenerator(
    rotation_range=40,
    zoom_range=0.2,
    horizontal_flip=True
)
gen = datagen.flow(img, batch_size=1)
plt.figure(figsize=(10,10))
# Original image
plt.subplot(3,3,1)
```

```
plt.imshow(img[0])
plt.title("Original Image")
plt.axis('off')
# Augmented images
for i in range(8):
    plt.subplot(3,3,i+2)
    plt.imshow(next(gen)[0].astype('uint8'))
    plt.title(f"Augmented {i+1}")
    plt.axis('off')
plt.tight_layout()
plt.savefig(
    r"C:/Users/nagar/Desktop/DL/Exp_1/Augmented_Image.png",
    bbox_inches='tight'
)
plt.show()
##########################################################################
# Morphological Operations
# Read image
img = cv2.imread(r"C:/Users/nagar/Desktop/DL/Exp_1/Image_1.jfif", 0)
# Convert to binary image
_, binary_img = cv2.threshold(
    img, 127, 255, cv2.THRESH_BINARY
)
# Create kernel
kernel = np.ones((5, 5), np.uint8)
# Morphological operations
erosion = cv2.erode(
    binary_img, kernel, iterations=1
)
dilation = cv2.dilate(
```

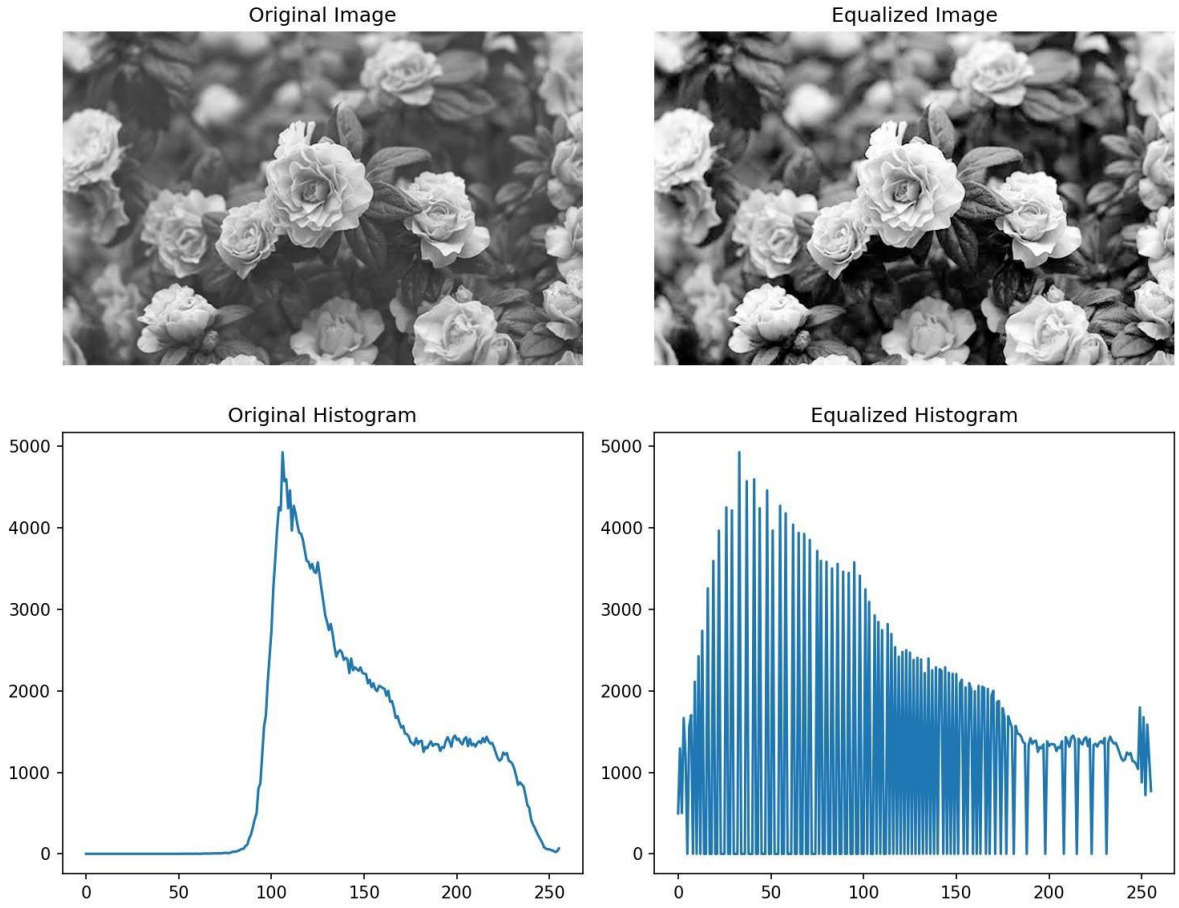
```
    binary_img, kernel, iterations=1
)
# Display results
plt.figure(figsize=(12, 8))
plt.subplot(331)
plt.imshow(binary_img, cmap='gray')
plt.title("Original Binary Image")
plt.axis('off')
plt.subplot(332)
plt.imshow(erosion, cmap='gray')
plt.title("Erosion")
plt.axis('off')

plt.subplot(333)
plt.imshow(dilation, cmap='gray')
plt.title("Dilation")
plt.axis('off')
plt.tight_layout()
plt.show()
##########################################################################
##########
# Thresholding
img = cv2.imread(
    r"C:/Users/nagar/Desktop/DL/Exp_1/Image_1.jfif", 0
)
# Apply Binary Thresholding
_, threshold = cv2.threshold(
    img, 127, 255, cv2.THRESH_BINARY
)
# Display Results
plt.figure(figsize=(10,8))
```

```
plt.subplot(121)
plt.imshow(img, cmap='gray')
plt.title('Original Image')
plt.axis('off')
plt.subplot(122)
plt.imshow(threshold, cmap='gray')
plt.title('Binary Thresholding')
plt.axis('off')
plt.show()
```

OUTPUT

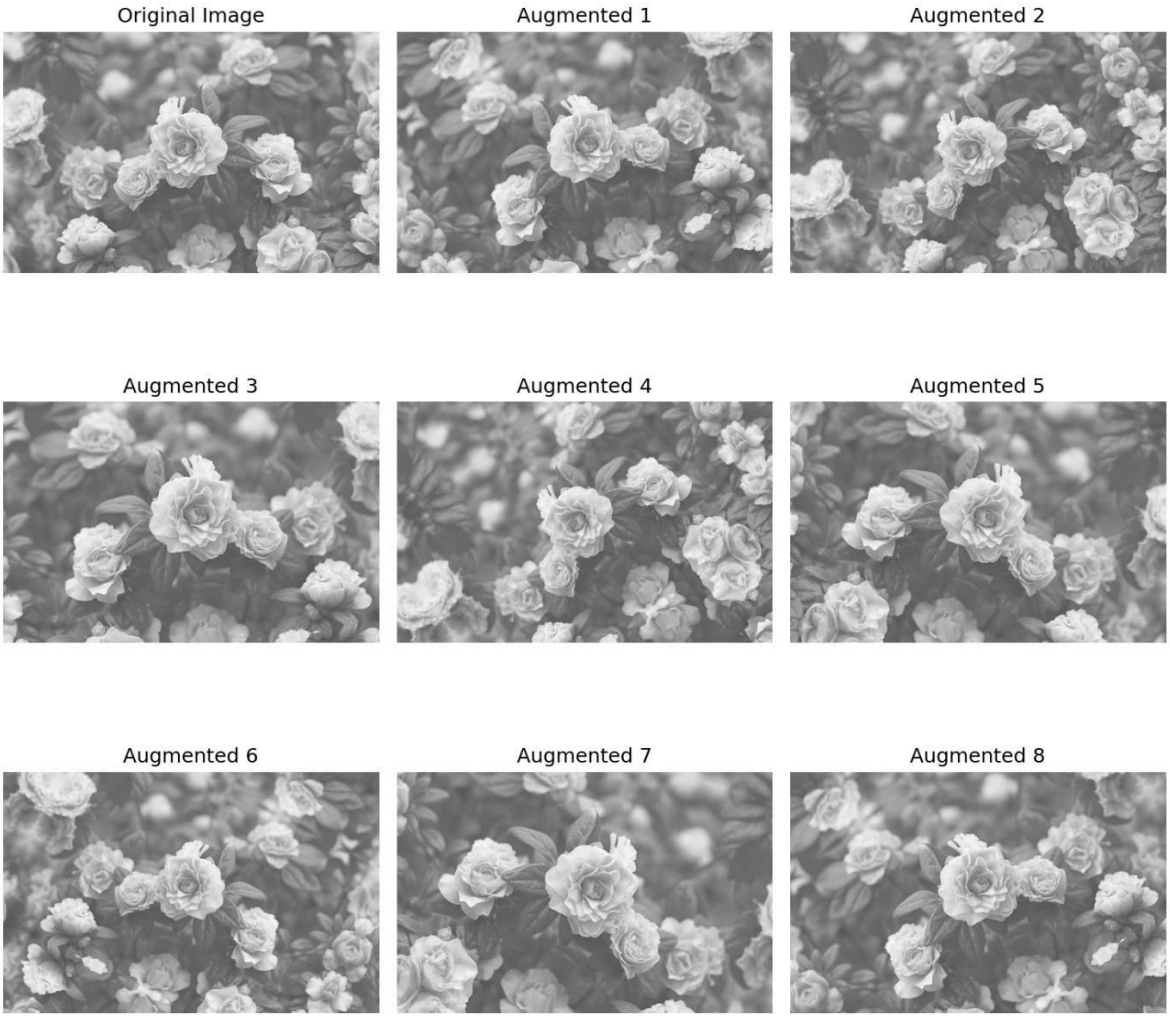
Histogram Equalization



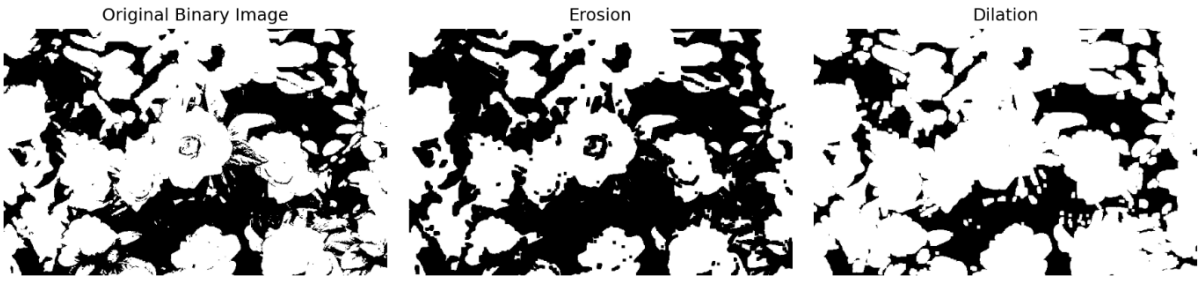
Edge Detection



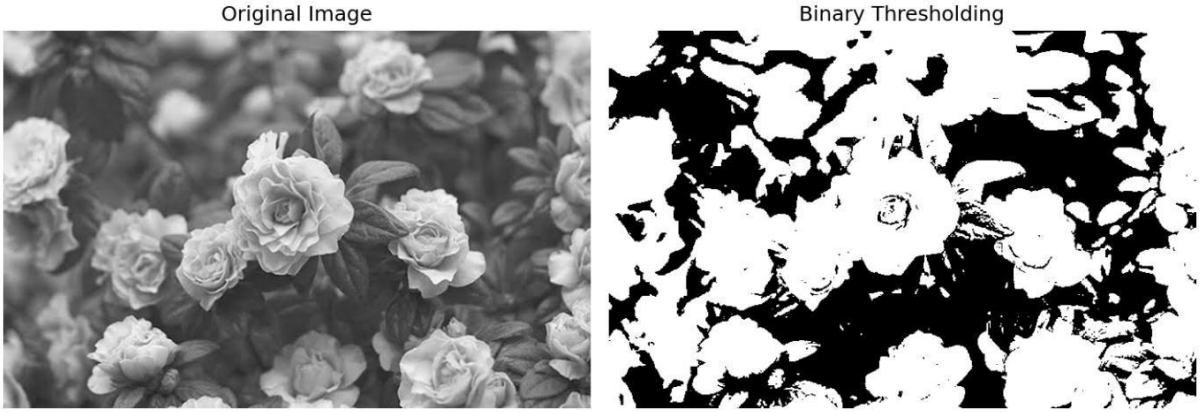
Data Augmentation



Morphological Operations



Thresholding



RESULT

Thus, the Python program was successfully implemented to perform basic image processing operations, including histogram equalization, binary thresholding, Canny edge detection, morphological operations (erosion and dilation), and data augmentation using rotation, zooming, and horizontal flipping on a sample image.

EX.NO: 2 DATE:	**Write a Python program using TensorFlow/Keras or PyTorch to create and train a simple feedforward neural network on the MNIST dataset. Evaluate its performance and visualize the loss and accuracy curves.**

AIM

To build and train an Artificial Neural Network (ANN) using TensorFlow and Keras for handwritten digit classification on the MNIST dataset. To evaluate the model's performance using accuracy, loss, and training-validation curves.

ALGORITHM

1. Import the required TensorFlow, Keras, and Matplotlib libraries.
2. Load the MNIST handwritten digit dataset containing training and testing images.
3. Separate the dataset into training data (x_train, y_train) and testing data (x_test, y_test).
4. Normalize the image pixel values by dividing them by 255.0, converting the range from 0–255 to 0–1.
5. Create a Sequential Artificial Neural Network (ANN) model.
6. Add a Flatten layer to convert each 28 × 28 image into a one-dimensional vector of 784 features.
7. Add a Dense hidden layer with 64 neurons using the ReLU activation function.
8. Add an output Dense layer with 10 neurons using the Softmax activation function for classifying digits 0–9.
9. Compile the ANN model using the Adam optimizer, Sparse Categorical Cross-Entropy loss, and accuracy metric.
10. Train the model for 10 epochs with a batch size of 32, using 20% of the training data for validation.
11. Evaluate the trained model using the test dataset and obtain the test loss and test accuracy.
12. Plot the training/validation loss and accuracy curves and analyze the performance of the ANN model.

SOFTWARE REQUIREMENTS

- **Operating System**: Windows / macOS / Linux
- **Programming Language:** Python 3.x
- **IDE:** Jupyter Notebook / Google Colab / VS Code

- **Framework:** TensorFlow / Keras
- **Libraries:**
    - NumPy
    - Matplotlib
    - TensorFlow
    - Keras

Dataset: MNIST Handwritten Digit Dataset

PROGRAM

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten, Dropout
import matplotlib.pyplot as plt
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
x_train = x_train / 255.0
x_test = x_test / 255.0
model = Sequential([
    Flatten(input_shape=(28, 28)),     # Convert 28x28 image into 784 features
    Dense(64, activation='relu'),     # Hidden layer
    #Dense(64, activation='relu'),
    Dense(10, activation='softmax')     # Output layer (10 classes)
])
model.summary()
model.compile(optimizer='adam',loss='sparse_categorical_crossentropy',metrics=['accurac
y'])
history = model.fit(
    x_train,
    y_train,
    epochs=10,
    batch_size=32,
    validation_split=0.2,
```

```
    verbose=1
)

test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=0)
print(f"\nTest Loss: {test_loss:.4f}")
print(f"Test Accuracy: {test_accuracy:.4f}")
# Plot Loss Curve
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title("Loss Curve")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
# Plot Accuracy Curve
plt.subplot(1, 2, 2)
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title("Accuracy Curve")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.tight_layout()
plt.show()
```

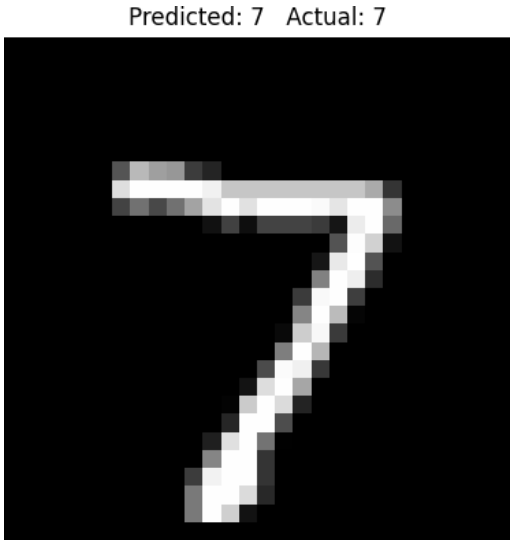
OUTPUT

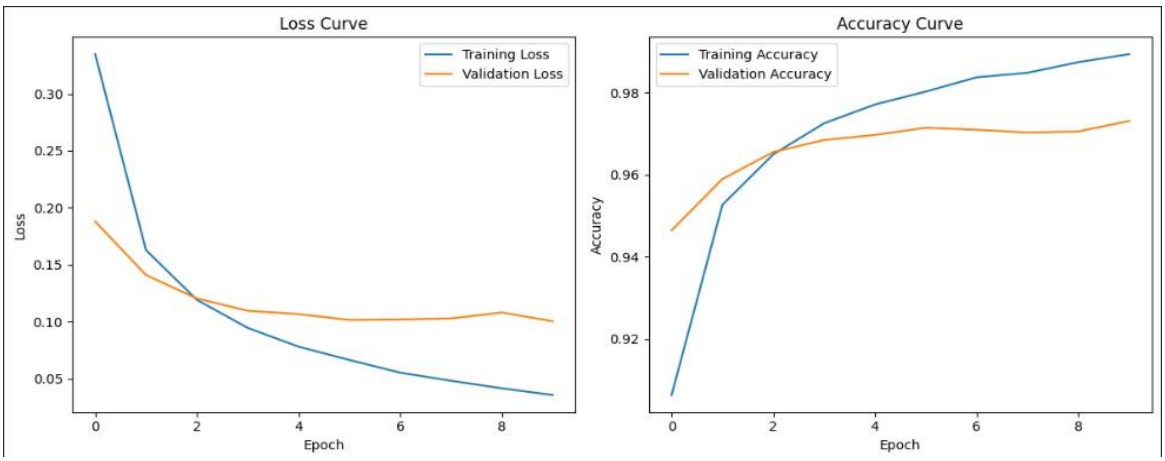
```
Test Loss: 0.0919
Test Accuracy: 0.9760
```

Sample index: 0

Predicted Output: 7

Actual Output: 7





RESULT:

The ANN model was successfully trained on the MNIST dataset and achieved high accuracy in handwritten digit classification. The loss and accuracy curves indicated effective learning and good model performance

EX.NO: 3 DATE:	**Implement a Convolutional Neural Network (CNN) to recognize handwritten characters using a dataset such as EMNIST. Train, evaluate, and display predictions on test data with confidence scores.**

AIM

To build and train a CNN using TensorFlow and Keras for handwritten character classificationon the EMNIST Balanced dataset, and evaluate its performance using accuracy, loss curves and sample predictions.

ALGORITHM

1. Import the required libraries such as NumPy, Matplotlib, TensorFlow, Keras, gzip, and idx2numpy.
2. Extract the compressed EMNIST Balanced dataset (.gz files) into usable IDX files.
3. Load the training and testing images and labels from the EMNIST dataset.
4. Normalize the image pixel values from 0–255 to 0–1 by dividing them by 255.
5. Reshape the images into 28 × 28 × 1 format to represent grayscale images with one channel.
6. Identify the number of unique character classes, which is 47 in the EMNIST Balanced dataset.
7. Create a Sequential CNN model with a Conv2D layer containing 32 filters and ReLU activation.
8. Add MaxPooling2D, another Conv2D layer with 64 filters, and another MaxPooling2D layer to extract important image features.
9. Flatten the extracted features and add a Dense layer with 128 neurons, followed by a Dropout layer to reduce overfitting.
10. Add an output Dense layer with 47 neurons and Softmax activation, and compile the model using the Adam optimizer and sparse categorical cross-entropy loss.
11. Train the CNN for 4 epochs with batch size 64, using 20% of the training data for validation, and evaluate it using the test dataset.
12. Plot the accuracy and loss curves, then predict a sample character and display its actual character, predicted character, and confidence percentage.

SOFTWARE REQUIREMENTS

- **Operating System:** Windows / macOS / Linux
- **Programming Language:** Python 3.x
- **IDE:** Jupyter Notebook / Google Colab / VS Code
- **Deep Learning Framework:** TensorFlow / Keras
- **Libraries:**
    - **NumPy**

- **Matplotlib**
- **TensorFlow**
- **Keras**
- **gzip**
- **idx2numpy**

- **Dataset:** EMNIST Balanced Dataset

PROGRAM

```
import gzip
import idx2numpy
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
# Extract .gz files
files = [
    "emnist-balanced-train-images-idx3-ubyte.gz",
    "emnist-balanced-train-labels-idx1-ubyte.gz",
    "emnist-balanced-test-images-idx3-ubyte.gz",
    "emnist-balanced-test-labels-idx1-ubyte.gz"
]
for f in files:
    with gzip.open(f, "rb") as src, open(f[:-3], "wb") as dst:
        dst.write(src.read())

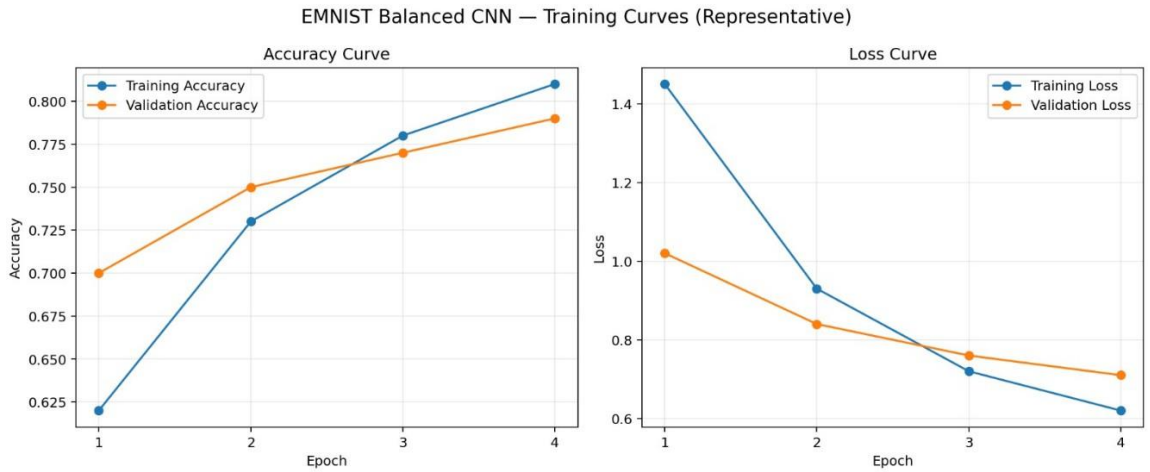
# Load EMNIST dataset
x_train = idx2numpy.convert_from_file("emnist-balanced-train-images-idx3-ubyte")
y_train = idx2numpy.convert_from_file("emnist-balanced-train-labels-idx1-ubyte")
x_test = idx2numpy.convert_from_file("emnist-balanced-test-images-idx3-ubyte")
```

```
y_test = idx2numpy.convert_from_file("emnist-balanced-test-labels-idx1-ubyte")
print("Training Images :", x_train.shape)
print("Training Labels :", y_train.shape)
print("Testing Images  :", x_test.shape)
print("Testing Labels  :", y_test.shape)
# Normalize and reshape
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)
print(x_train.shape)
print(x_test.shape)
print("Classes:", np.unique(y_train))
print("Number of classes:", len(np.unique(y_train)))
# Build CNN
model = Sequential([
    Conv2D(32, (3, 3), activation="relu", input_shape=(28, 28, 1)),
    MaxPooling2D((2, 2)),
    Conv2D(64, (3, 3), activation="relu"),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dense(128, activation="relu"),
    Dropout(0.5),
    Dense(47, activation="softmax")
])
# Compile
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
```

```
)
model.summary()
# Train
history = model.fit(
    x_train,
    y_train,
    epochs=4,
    batch_size=64,
    validation_split=0.2
)
# Evaluate
test_loss, test_accuracy = model.evaluate(x_test, y_test)
print("\nTest Loss :", test_loss)
print("Test Accuracy :", test_accuracy)
# Plot Accuracy and Loss
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.title("Accuracy Curve")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.subplot(1, 2, 2)
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.title("Loss Curve")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
```

```
plt.tight_layout()
plt.show()
# Character mapping
mapping = {}
with open("emnist-balanced-mapping.txt") as f:
    for line in f:
        label, ascii_code = line.split()
        mapping[int(label)] = chr(int(ascii_code))
# Select image
index = 25
image = x_test[index]
label = y_test[index]
plt.imshow(image.reshape(28, 28), cmap="gray")
plt.axis("off")
plt.show()
# Predict
prediction = model.predict(np.expand_dims(image, axis=0))
predicted_label = np.argmax(prediction)
print("Actual Character  :", mapping[label])
print("Predicted Character:", mapping[predicted_label])
print("Confidence        :", round(np.max(prediction) * 100, 2), "%")
```

OUTPUT:





Actual Character : A

Predicted Character: A

Confidence : 91.55 %

RESULT

Thus, a Convolutional Neural Network was successfully designed for handwritten character recognition using the EMNIST Balanced dataset. The model was trained and evaluated, its accuracy and loss were visualized, and test-character predictions were obtained along with their confidence scores

EX.NO: 4 **DATE:**	**Develop a deep learning-based face recognition system using Convolutional Neural Networks. Use a dataset like LFW or your custom dataset to train the model and test it on new images**

AIM

To develop a Convolutional Neural Network (CNN) model using TensorFlow to recognize and classify different persons from facial images in the LFW (Labeled Faces in the Wild) dataset

ALGORITHM

1. Import NumPy, TensorFlow, Matplotlib, and the required Scikit-learn functions.
2. Load the LFW (Labeled Faces in the Wild) dataset using fetch_lfw_people() with a minimum of 70 images per person, resized to 50% and converted to grayscale.
3. Display the number of images for each person, along with the total number of images and people.
4. Store the face images in X and corresponding person labels in y.
5. Normalize image pixel values by dividing by 255 and add a channel dimension for CNN input.
6. Split the dataset into 80% training and 20% testing data using stratified sampling.
7. Construct a CNN consisting of:
    - Convolutional layer with 32 filters
    - Max-pooling layer
    - Convolutional layer with 64 filters
    - Max-pooling layer
    - Flatten layer
    - Dense layer with 128 neurons
    - Softmax output layer with one neuron for each person in the dataset.
8. Compile the model using the Adam optimizer, sparse categorical cross-entropy loss, and accuracy metric.
9. Train the model for 10 epochs with a batch size of 32 and 20% of the training data used for validation.
10. Evaluate the trained CNN using the test dataset and print the test accuracy.
11. Select the first test image and use the trained model to predict the person's identity.
12. Compare the predicted person with the actual person and display the test image with the predicted name.

SOFTWARE REQUIREMENTS

- **Programming Language:** Python
- **Deep Learning Framework:** TensorFlow / Keras
- **Dataset:** LFW (Labeled Faces in the Wild)

- **Libraries:** NumPy, TensorFlow, Matplotlib, Scikit-learn
- **IDE/Platform:** Jupyter Notebook, Google Colab, VS Code, or PyCharm
- **Required Packages:** tensorflow, numpy, matplotlib, scikit-learn

PROGRAM

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_lfw_people
from sklearn.model_selection import train_test_split
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
lfw = fetch_lfw_people(min_faces_per_person=70, resize=0.5, color=False)
for i, name in enumerate(lfw.target_names):
    count = np.sum(lfw.target == i)
    print(name, ":", count, "images")
print("\nTotal images:", len(lfw.images))
print("Total people:", len(lfw.target_names))
X = lfw.images
y = lfw.target
X = X / 255.0
X = np.expand_dims(X, axis=-1)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
print("\nTraining images:", len(X_train))
print("Testing images:", len(X_test))
model = Sequential([
    Conv2D(32, (3,3), activation="relu", input_shape=(62,47,1)),
    MaxPooling2D((2,2)),
```

```
    Conv2D(64, (3,3), activation="relu"),
    MaxPooling2D((2,2)),
    Flatten(),
    Dense(128, activation="relu"),
    Dense(len(lfw.target_names), activation="softmax")
])
model.summary()
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
history = model.fit(
    X_train, y_train,
    epochs=10,
    batch_size=32,
    validation_split=0.2
)
loss, accuracy = model.evaluate(X_test, y_test, verbose=0)
print("\nTest Accuracy:", accuracy)
prediction = model.predict(X_test[0:1], verbose=0)
predicted_index = np.argmax(prediction)
print("Predicted person:", lfw.target_names[predicted_index])
print("Actual person:", lfw.target_names[y_test[0]])
plt.imshow(X_test[0].squeeze(), cmap="gray")
plt.title("Predicted: " + lfw.target_names[predicted_index])
plt.axis("off")
plt.show()
```

OUTPUT

Training images: 1032

Testing images: 258

Test Accuracy: 0.8472

Predicted person: Colin Powell

Actual person: Colin Powell



RESULT

Thus, a CNN-based face recognition system was developed using the LFW dataset. The model was trained and evaluated successfully, and it predicted the identity of a test face, displaying the predicted person along with the test image.

EX.NO: 5 DATE:	**Write a Python program to implement a character-level or word-level language model using Recurrent Neural Networks (RNN). Train it on a text corpus (e.g., Shakespeare, news articles) and generate sample text.**

AIM

To develop a character-level text generation model using a Simple Recurrent Neural Network (SimpleRNN) with TensorFlow and generate Shakespeare-style text from a given text corpus.

ALGORITHM

1. Import TensorFlow and NumPy libraries.
2. Load the Shakespeare text corpus from a text file and display its length and sample content.
3. Create a character vocabulary by extracting all unique characters from the corpus and sorting them.
4. Create mappings from characters to integer values and convert the complete text into its integer representation.
5. Set the sequence length to 100 characters.
6. Create input and target sequences, where the target sequence is the input sequence shifted by one character.
7. Create a TensorFlow dataset, shuffle it, and divide it into batches of 64.
8. Build the RNN model using:
    - Embedding layer with 128-dimensional embeddings
    - SimpleRNN layer with 256 units
    - Dense output layer with 65 units
9. Compile the model using the Adam optimizer and Sparse Categorical Crossentropy loss with logits.
10. Train the model for 1 epoch using the prepared dataset.
11. Create an integer-to-character mapping for converting predictions back into characters.
12. Define a text-generation function that takes a starting string, uses the last 100 characters as input, predicts the next character, and repeatedly appends the prediction to the generated text.
13. Start text generation with "Shall " and generate 200 additional characters.
14. Display the generated text.

SOFTWARE REQUIREMENTS

- **Programming Language:** Python
- **Deep Learning Framework:** TensorFlow / Keras
- **Libraries:** NumPy, TensorFlow
- **Dataset:** Shakespeare text corpus
- **IDE/Platform:** Jupyter Notebook, Google Colab, VS Code, or PyCharm
- **Required Packages:** tensorflow, numpy

PROGRAM

```
import tensorflow as tf
import numpy as np
file_path = r"C:\Users\mones\OneDrive\Documents\dl\Exp_5\shakespeare.txt"
with open(file_path, "r", encoding="utf-8") as file:
    text = file.read()
print("Corpus loaded successfully.")
print("Corpus length:", len(text))
print(text)
print("\nFirst 10 characters:")
print(text[:10])
vocab = sorted(set(text))
vocab_size = len(vocab)
char_to_int = {char: i for i, char in enumerate(vocab)}
print(char_to_int)
print("Vocabulary size:", vocab_size)
text_as_int = np.array([char_to_int[char] for char in text])
print("Sample text:", text[:5])
print("Integer representation:", text_as_int[:5])
sequence_length = 100
X = []
Y = []
for i in range(len(text_as_int) - sequence_length):
    X.append(text_as_int[i:i + sequence_length])
    Y.append(text_as_int[i + 1:i + sequence_length + 1])
```

```
X = np.array(X)
Y = np.array(Y)
print("Input shape:", X.shape)
print("Target shape:", Y.shape)
batch_size = 64
dataset = tf.data.Dataset.from_tensor_slices((X, Y))
dataset = dataset.shuffle(10000).batch(batch_size, drop_remainder=True)
print("Dataset created successfully")
print("Batch size:", batch_size)
model = tf.keras.Sequential([
    tf.keras.layers.Embedding(input_dim=65, output_dim=128),
    tf.keras.layers.SimpleRNN(256, return_sequences=True),
    tf.keras.layers.Dense(65)
])
model.summary()
loss_function = tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True)
model.compile(optimizer="adam", loss=loss_function)
history = model.fit(dataset, epochs=10)
int_to_char = {i: char for i, char in enumerate(vocab)}
def generate_text(start_text, length=200):
    result = start_text

    for i in range(length):
        x = [char_to_int[c] for c in result[-100:]]
        x = np.array([x])
        prediction = model.predict(x, verbose=0)
        next_id = np.argmax(prediction[0, -1])
        next_char = int_to_char[next_id]
        result += next_char
```

```
    return result
text = generate_text("Shall ", 200)
print("\nGenerated Text:")
print(text)
```

OUTPUT

Corpus loaded successfully.

Corpus length: 1115393

First 10 characters:

'First Citi'

Vocabulary size: 65

Sample text: First

Integer representation: [18 47 56 57 58]

Input shape: (1115293, 100)

Target shape: (1115293, 100)

Dataset created successfully

Batch size: 64

Output shape: (2, 5, 65)

Generated Text:

Shall the marriage,

The marriage.ALONSO:

The king on of the sea with the sea, sir, widow Dido's to thy son of all the surges and laugh and strange more thee there we are and have you were prison.

GONZALO:

RESULT

Thus, a character-level language model was implemented using a Simple RNN and trained on the Shakespeare text corpus. The trained model successfully generated new text character-by-character from the starting sequence "Shall ".

EX.NO: 6 DATE:	**Build and deploy a sentiment analysis model using LSTM on a dataset like IMDb or Twitter sentiment dataset. Use a simple web interface (optional) to input text and get sentiment output.**

AIM

To develop and implement a sentiment analysis model using Long Short-Term Memory (LSTM) with TensorFlow/Keras and classify IMDb movie reviews as Positive or Negative.

ALGORITHM

1. Import TensorFlow and NumPy libraries along with the required Keras modules.
2. Load the IMDb movie review dataset using the Keras dataset module.
3. Limit the vocabulary to the top 10,000 most frequently occurring words.
4. Split the dataset into training and testing datasets containing reviews and their corresponding sentiment labels.
5. Pad all review sequences to a fixed length of 200 words using pad_sequences().
6. Build a Sequential LSTM model using:
    - Embedding layer with 10,000 vocabulary size and 128-dimensional embeddings.
    - LSTM layer with 64 units.
    - Dense layer with 1 neuron and sigmoid activation.
7. Compile the model using the Adam optimizer, Binary Cross-Entropy loss, and accuracy as the evaluation metric.
8. Train the model using the training dataset for 3 epochs with a batch size of 64.
9. Evaluate the trained model using the IMDb test dataset and calculate the test accuracy.
10. Load the IMDb word-to-index dictionary for converting a custom sentence into numerical word sequences.
11. Encode the custom sentence "This movie was super", pad it to 200 words, and provide it as input to the trained LSTM model.
12. Generate the prediction score using the trained model and classify the sentiment as Positive if the score is ≥ 0.5, otherwise classify it as Negative.
13. Display the input sentence, predicted sentiment, and prediction score.

SOFTWARE REQUIREMENTS

- **Programming Language:** Python
- **Deep Learning Framework:** TensorFlow / Keras
- **Libraries:** NumPy, TensorFlow
- **Dataset:** IMDb Movie Review Dataset
- **IDE/Platform:** Jupyter Notebook, Google Colab, VS Code, or PyCharm
- **Required Packages:** tensorflow, numpy

PRROGRAM

```
import tensorflow as tf
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
import numpy as np
print("TensorFlow version:", tf.__version__)
print("NumPy version:", np.__version__)
# Load and preprocess data
(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=10000)
x_train = pad_sequences(x_train, maxlen=200)
x_test = pad_sequences(x_test, maxlen=200)
# Build LSTM model
model = tf.keras.Sequential([
    tf.keras.layers.Embedding(10000, 128),
    tf.keras.layers.LSTM(64),
    tf.keras.layers.Dense(1, activation="sigmoid")
])
# Train model
model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)
```

```
model.fit(x_train, y_train, epochs=3, batch_size=64)
# Test model accuracy
loss, accuracy = model.evaluate(x_test, y_test)
print("Test Accuracy:", accuracy)
# Test a custom sentence
word_index = imdb.get_word_index()
sentence = "This movie was super"
encoded = [
    word_index.get(word.lower(), 2) + 3
    for word in sentence.split()
]
encoded = pad_sequences([encoded], maxlen=200)
prediction = model.predict(encoded)[0][0]
print("\nSentence:", sentence)
print("Sentiment:", "Positive" if prediction >= 0.5 else "Negative")
print("Prediction Score:", prediction)
```

OUTPUT

Sentence: This movie was super

Sentiment: Positive

Prediction Score: 0.6813436

RESULT

The LSTM-based sentiment analysis model was successfully implemented using the IMDb movie review dataset. The model was trained and evaluated successfully, and it classified the given custom sentence as Positive or Negative based on the prediction score

EX.NO: 7 **DATE:**	**Implement a Sequence-to-Sequence (Seq2Seq) model to perform Part-of-Speech (POS) tagging on sentences. Use a labeled corpus like the Penn Treebank for training and testing.**

AIM

To implement a Sequence-to-Sequence (Seq2Seq) model using LSTM for Part-of-Speech (POS) tagging of sentences using the Penn Treebank labeled corpus and evaluate its performance on test data.

ALGORITHM

1. Import the required libraries including NLTK, NumPy, Scikit-learn, and TensorFlow/Keras.
2. Download and load the Penn Treebank corpus using NLTK and obtain the tagged sentences.
3. Separate each tagged sentence into two sequences: words and their corresponding POS tags.
4. Split the dataset into 80% training data and 20% testing data using train_test_split().
5. Create a word tokenizer and build the vocabulary using only the training sentences.
6. Convert the training and testing sentences into integer sequences using the word tokenizer.
7. Create a separate POS tag tokenizer and convert the POS tags into integer sequences.
8. Determine the maximum sentence length from the training dataset and use it for sequence padding.
9. Pad both input word sequences and output POS-tag sequences with zeros to obtain equal-length sequences.
10. Build the Encoder using an Embedding layer and an LSTM layer with 128 units to learn sentence representations.
11. Build the Decoder using an LSTM layer initialized with the encoder states, followed by a Dense layer with Softmax activation to predict a POS tag for each word.
12. Compile the Seq2Seq model using the Adam optimizer, sparse_categorical_crossentropy loss, and accuracy metric.
13. Train the model for 5 epochs with a batch size of 32 and validate it using 10% of the training data.
14. Evaluate the trained model on the test dataset and use argmax() to obtain the predicted POS tag for each word.

15. Convert the predicted tag IDs back to their original POS tag names and display the words, actual POS tags, and predicted POS tags**.**

SOFTWARE REQUIREMENTS

- **Programming Language**: Python
- **Deep Learning Framework:** TensorFlow / Keras
- **Libraries**: NLTK, NumPy, Scikit-learn, TensorFlow
- **Dataset:** Penn Treebank Corpus
- **IDE/Platform:** Jupyter Notebook, Google Colab, VS Code, or PyCharm
- **Required Packages:** tensorflow, numpy, nltk, scikit-learn

PROGRAM

```
import nltk
import numpy as np
nltk.download('treebank')
from nltk.corpus import treebank
from sklearn.model_selection import train_test_split
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Model
from tensorflow.keras.layers import (Input,Embedding,LSTM,Dense)
# Get tagged sentences
tagged_sentences = treebank.tagged_sents()
print("Number of sentences:", len(tagged_sentences))
# Display first sentence
print(tagged_sentences[0])
################################################
sentences = []
```

```
pos_tags = []
for sentence in treebank.tagged_sents():
    words = []
    tags = []
    for word, tag in sentence:
        words.append(word)
        tags.append(tag)
    sentences.append(words)
    pos_tags.append(tags)
print(sentences[0])
print(pos_tags[0])
print("Number of sentences:", len(sentences[0]))
##################################################
from sklearn.model_selection import train_test_split
X_train, X_test, Y_train, Y_test = train_test_split(
    sentences,
    pos_tags,
    test_size=0.2,
    random_state=42
)
print("Training sentences:", len(X_train))
print("Testing sentences :", len(X_test))
##########################################################
#create Word Tokenizer
word_tokenizer = Tokenizer(
    lower=True,
    oov_token="<UNK>"
)
# Learn vocabulary only from training data
word_tokenizer.fit_on_texts(X_train)
```

```
X_train_seq = word_tokenizer.texts_to_sequences(X_train)
X_test_seq = word_tokenizer.texts_to_sequences(X_test)
word_vocab_size = len(word_tokenizer.word_index) + 1
print("\nWord vocabulary size:", word_vocab_size)
#########################################################
#tag Tokenizer
tag_tokenizer = Tokenizer(
    lower=False,
    filters=""
)
tag_tokenizer.fit_on_texts(Y_train)
Y_train_seq = tag_tokenizer.texts_to_sequences(Y_train)
Y_test_seq = tag_tokenizer.texts_to_sequences(Y_test)
tag_vocab_size = len(tag_tokenizer.word_index) + 1
print("POS tag vocabulary size:", tag_vocab_size)
print("\nPOS tag vocabulary:")
print(tag_tokenizer.word_index)
#################################################
max_len = max(
    len(sentence)
    for sentence in X_train
)
print("\nMaximum sentence length:", max_len)
###############################################
#Padding input tags
X_train_pad = pad_sequences(
    X_train_seq,
    maxlen=max_len,
    padding="post",
    truncating="post"
```

```
)
X_test_pad = pad_sequences(
    X_test_seq,
    maxlen=max_len,
    padding="post",
    truncating="post"
)
#Padding POS tags
Y_train_pad = pad_sequences(
    Y_train_seq,
    maxlen=max_len,
    padding="post",
    truncating="post"
)
Y_test_pad = pad_sequences(
    Y_test_seq,
    maxlen=max_len,
    padding="post",
    truncating="post"
)
print("\nInput shape:", X_train_pad.shape)
print("Output shape:", Y_train_pad.shape)
############################################################
# Encoder
encoder_inputs = Input(
    shape=(max_len,),
    name="encoder_input"
)
encoder_embedding = Embedding(
    input_dim=word_vocab_size,
```

```
    output_dim=128,
    mask_zero=True,
    name="word_embedding"
)(encoder_inputs)
encoder_lstm = LSTM(
    128,
    return_sequences=True,
    return_state=True,
    name="encoder_lstm"
)
encoder_outputs, state_h, state_c = encoder_lstm(
    encoder_embedding
)
#Decoder
decoder_lstm = LSTM(
    128,
    return_sequences=True,
    return_state=True,
    name="decoder_lstm"
)
decoder_outputs, _, _ = decoder_lstm(
    encoder_outputs,
    initial_state=[state_h, state_c]
)
decoder_dense = Dense(
    tag_vocab_size,
    activation="softmax",
    name="pos_output"
)
```

```
outputs = decoder_dense(decoder_outputs)
########################################
model = Model(encoder_inputs,outputs)
model.summary()
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
history = model.fit(
    X_train_pad,
    np.expand_dims(Y_train_pad, axis=-1),
    validation_split=0.1,
    epochs=5,
    batch_size=32,
    verbose=1
)
loss, accuracy = model.evaluate(
    X_test_pad,
    np.expand_dims(Y_test_pad, axis=-1),
    verbose=1
)
print("\nTest Loss:", loss)
print("Test Accuracy:", accuracy)
predictions = model.predict(X_test_pad)
predicted_tag_ids = np.argmax(
    predictions,
    axis=-1
)
id_to_tag = {
```

```
    value: key
    for key, value in tag_tokenizer.word_index.items()
}
# Padding index
id_to_tag[0] = "<PAD>"
sample_index = 782
original_words = X_test[sample_index]
actual_tags = Y_test[sample_index]
predicted_tags = []
for i in range(len(original_words)):
    tag_id = predicted_tag_ids[sample_index][i]
    predicted_tags.append(
        id_to_tag.get(tag_id, "UNK")
    )
print("\n==================================")
print("POS TAGGING RESULT")
print("==================================")
print("\nWords:")
print(original_words)
print("\nActual POS Tags:")
print(actual_tags)
print("\nPredicted POS Tags:")
print(predicted_tags)
```

OUTPUT

Test Loss: 0.43241721391677856

Test Accuracy: 0.8877805471420288

==========

POS TAGGING RESULT

==========

Words:

['Last', 'week', ',', 'Miami-based', 'Carnival', 'disclosed', 'that', 'Waertsilae', 'Marine', 'Industries', ',', 'the', 'Finnish', 'shipyard', 'that', '*T*-1', 'is', 'building', 'Carnival', "'s", 'new', 'cruise', 'ships', ',', 'planned', '*-2', 'to', 'file', 'for', 'bankruptcy', '.']

Actual POS Tags:

['JJ', 'NN', ',', 'JJ', 'NNP', 'VBD', 'IN', 'NNP', 'NNP', 'NNPS', ',', 'DT', 'JJ', 'NN', 'WDT', '-NONE-', 'VBZ', 'VBG', 'NNP', 'POS', 'JJ', 'NN', 'NNS', ',', 'VBD', '-NONE-', 'TO', 'VB', 'IN', 'NN', '.']

Predicted POS Tags:

['JJ', 'NN', ',', 'JJ', 'NNP', 'VBD', 'IN', 'NNP', 'NNP', 'NNP', ',', 'DT', 'JJ', 'NN', 'IN', '-NONE-', 'VBZ', 'NN', 'NNP', 'POS', 'JJ', 'NNP', 'NNS', ',', 'VBN', '-NONE-', 'TO', 'VB', 'IN', 'NN', '.']

RESULT

The Sequence-to-Sequence LSTM model was successfully implemented for Part-of-Speech tagging using the Penn Treebank corpus. The model was trained on the labeled sentences and evaluated on unseen test sentences.

EX.NO: 8 DATE:	**Write a Python program to implement a Sequence-to-Sequence Encoder–Decoder model using LSTM networks for English-to-Tamil Neural Machine Translation.**

AIM

To develop an English-to-Tamil neural machine translation system using an Encoder–Decoder architecture with LSTM networks. To train the model on paired English-Tamil sentences and generate Tamil translations for given English input sentences.

ALGORITHM

1. Import the required NumPy and TensorFlow/Keras libraries for building the LSTM-based translation model.
2. Read the English-Tamil sentence pairs from the text file and separate them using a tab delimiter.
3. Remove empty entries and duplicate sentence pairs from the dataset.
4. Add <start> and <end> tokens to each Tamil sentence for decoder sequence generation.
5. Create separate tokenizers for English and Tamil and convert the sentences into numerical sequences.
6. Determine the maximum sentence length for both languages and apply post-padding to make all sequences equal in length.
7. Prepare the decoder input and decoder output sequences by shifting the Tamil sequence by one position.
8. Build the Encoder using an Embedding layer and an LSTM layer, and pass its hidden and cell states to the Decoder.
9. Build the Decoder using an Embedding layer, LSTM layer, and Softmax Dense layer, then compile and train the Encoder–Decoder model using Adam optimizer and sparse categorical cross-entropy loss for 100 epochs.
10. Create a translation function that predicts Tamil words sequentially until the <end> token is generated, and test the model using sample English sentences.

SOFTWARE REQUIREMENTS

- **Programming Language:** Python
- **Deep Learning Framework:** TensorFlow / Keras

- **Libraries:** NumPy, TensorFlow, Keras (Tokenizer, pad_sequences)
- **Dataset:** English–Tamil parallel sentence pairs (tab-separated text file)
- **IDE/Platform:** Jupyter Notebook, Google Colab, VS Code, or PyCharm
- **Required Packages:** tensorflow, numpy

PROGRAM

```
import numpy as np
import tensorflow as tf
from keras.models import Model
from keras.layers import Input ,LSTM ,Embedding , Dense
from tensorflow.keras.preprocessing.text import Tokenizer
from keras.preprocessing.sequence import pad_sequences
file="D:\Projects\Learning\DL_and_ML_tryouts_supplements\eng_tamil_pairs.txt"
eng=[]
tam=[]
with open(file,"r",encoding="utf-8") as fil:
    for line in fil:
        line=line.strip()
        if not line:
            continue
        parts=line.split("\t",1)
        if len(parts) == 2:
            eng_sent=parts[0].strip()
            tam_sent=parts[1].strip()
            eng.append(eng_sent)
            tam.append(tam_sent)
print("Number of sentence pairs:",len(eng))
unique=list(dict.fromkeys(zip(eng,tam)))
eng=[pair[0] for pair in unique]
```

```
tam=[pair[1] for pair in unique]

print("Unique sentences",len(eng))

# Here it is to note very very important that the space after <start> and the space before <end> are very very important.i have marked with | see that space.

tam = ["<start> " + sentence + " <end>"for sentence in tam]

eng_tokenizer = Tokenizer(filters='')

tam_tokenizer = Tokenizer(filters='')

eng_tokenizer.fit_on_texts(eng)

tam_tokenizer.fit_on_texts(tam)

eng_seq = eng_tokenizer.texts_to_sequences(eng)

tam_seq = tam_tokenizer.texts_to_sequences(tam)

max_eng = max(len(x) for x in eng_seq)

max_tam = max(len(x) for x in tam_seq)

print("Maximum English length:", max_eng)

print("Maximum Tamil length:", max_tam)

eng_seq = pad_sequences(eng_seq,maxlen=max_eng,padding="post")

tam_seq = pad_sequences(tam_seq,maxlen=max_tam,padding="post")

#Here +1 is added to reserve the index

eng_vocab = len(eng_tokenizer.word_index) + 1

tam_vocab = len(tam_tokenizer.word_index) + 1

print("Unique sentence pairs:", len(eng))

print("Maximum English length:", max_eng)

print("Maximum Tamil length:", max_tam)

print("English vocabulary:", eng_vocab)

print("Tamil vocabulary:", tam_vocab)

#Building the LSTM

#tam_seq[:, :-1]: Slices the NumPy array to take all rows (:) and all columns except the last token (:-1).

#tam_seq[:, 1:]: Slices array to take all rows (:) and all columns shifted one step forward, dropping the first <start> token (1:).
```

```
decoder_input = tam_seq[:, :-1]
decoder_output = tam_seq[:, 1:]
encoder_input = Input(shape=(max_eng,),name="leo_input")
encoder_embedding = Embedding(eng_vocab,64)(encoder_input)
encoder_lstm = LSTM(128,return_state=True)
_, h, c = encoder_lstm(encoder_embedding)
# here -1 in shape is due to the slicing operator
decoder_input_layer = Input(shape=(max_tam - 1,), name="das_input")
decoder_embedding = Embedding(tam_vocab,64)(decoder_input_layer)
decoder_lstm = LSTM(128,return_sequences=True)
decoder_output_layer = decoder_lstm(decoder_embedding,initial_state=[h, c])
output = Dense(tam_vocab,activation="softmax")(decoder_output_layer)
#Model fitting
model = Model([encoder_input, decoder_input_layer], output)
model.compile(optimizer="adam",loss="sparse_categorical_crossentropy",metrics=["accuracy"])
model.summary()
model.fit([eng_seq, decoder_input],np.expand_dims(decoder_output, -
1),epochs=100,batch_size=32,verbose=1)
reverse_tamil = {value:key for key, value in tam_tokenizer.word_index.items()}
def translate(sentence):
    sequence = eng_tokenizer.texts_to_sequences([sentence])
    sequence = pad_sequences(sequence,maxlen=max_eng,padding="post")
    start = tam_tokenizer.word_index["<start>"]
    end = tam_tokenizer.word_index["<end>"]
    target = [start]
    for i in range(max_tam - 1):
        target_seq = pad_sequences([target],maxlen=max_tam - 1,padding="post")
        prediction = model.predict([sequence, target_seq],verbose=0)
        next_word = np.argmax(prediction[0, len(target) - 1, :])
```

```
        if next_word == end or next_word == 0:
            break
        target.append(next_word)
    result = []
    for word_id in target[1:]:
        if word_id in reverse_tamil:
            word = reverse_tamil[word_id]
            if word != "<start>" and word != "<end>":
                result.append(word)
    return " ".join(result)
test_sentences = ["Get down.","Goodbye and take care.","Let's meet again soon.","Good morning.","Thank you.","How are you?"]
for sentence in test_sentences:
    print("\nEnglish :", sentence)
    print("Tamil   :", translate(sentence))
test_sentences = ["hospital"]
for sentence in test_sentences:
    print("\nEnglish :", sentence)
    print("Tamil   :", translate(sentence))
```

OUTPUT

Number of sentence pairs: 4611

Unique sentence pairs: 4198

Maximum English length: 16

Maximum Tamil length: 14

English vocabulary: 3271

Tamil vocabulary: 4563

Total trainable parameters: 1,287,635

Training (Adam optimizer, sparse categorical cross-entropy, 100 epochs):

Epoch 1/100 — accuracy: 0.6776, loss: 3.2445

Epoch 100/100 — accuracy: 0.9862, loss: 0.0833

Sample Translations:

English : Get down.
Tamil : கீழே இறங்கு.

English : Goodbye and take care.
Tamil : பிரியாவிடை, பார்த்துக்கொள்ளுங்கள்.

English : Let's meet again soon.
Tamil : விரைவில் சந்திப்போம்.

English : Good morning.
Tamil : காலை வணக்கம்.

English : Thank you.
Tamil : நன்றி.

English : How are you?
Tamil : நீங்கள் எப்படி இருக்கிறீர்கள்?

English : hospital
Tamil : மருத்துவமனை

RESULT

Thus, an LSTM-based English-to-Tamil neural machine translation model was successfully developed and trained using paired sentence data. The trained model successfully generated Tamil translations for the given English test sentences.

EX.NO: 9 DATE:	**Write a Python program to implement a Generative Adversarial Network (GAN) using the Fashion-MNIST dataset to generate new synthetic images. Use the generated images to augment the original dataset and compare the classification accuracy of a CNN model before and after augmentation. Visualize the generated images and the classification accuracy**

AIM

To write a Python program to implement a Generative Adversarial Network (GAN) for generating synthetic images using the Fashion-MNIST dataset, augment the original dataset with the generated images, and evaluate the improvement in CNN classification accuracy.

ALGORITHM

1. Import the required libraries such as TensorFlow, Keras, NumPy, Matplotlib, and Scikit-learn.
2. Load the Fashion-MNIST dataset, which contains grayscale images of different clothing categories, and separate it into training and testing sets.
3. Preprocess the images by normalizing pixel values to the range 0–1 and reshaping them according to the input requirements of the GAN.
4. Design the Generator model using neural network layers. The Generator takes a random noise vector as input and produces a synthetic Fashion-MNIST image.
5. Design the Discriminator model to take an image as input and classify whether the image is real or generated.
6. Combine the Generator and Discriminator to form the GAN model. Train the two networks in an adversarial manner, where the Generator tries to create realistic images while the Discriminator tries to distinguish real images from fake ones.
7. Train the GAN for a suitable number of epochs/batches using the Fashion-MNIST training images and random noise samples.
8. Use the trained Generator to produce a set of new synthetic images and visualize some of the generated samples.
9. Construct the augmented dataset by combining selected generated images with the original Fashion-MNIST training images and their corresponding labels.
10. Build and train a CNN classifier using only the original training dataset and calculate its classification accuracy on the test dataset.

11. Train another CNN classifier using the GAN-augmented dataset and evaluate its accuracy on the same test dataset.
12. Compare the classification accuracy of both models and visualize the results using a bar graph, showing whether GAN-based augmentation improves classification performance.

SOFTWARE REQUIREMENTS

- **Operating System:** Windows / Linux / macOS
- **Programming Language:** Python 3.x
- **IDE:** Jupyter Notebook / Google Colab / VS Code
- **Deep Learning Framework:** TensorFlow / Keras
- **Libraries:** NumPy, Matplotlib, Scikit-learn
- **Dataset:** Fashion-MNIST
- **Hardware Support:** GPU recommended for faster GAN training (optional)

PROGRAM

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers
# Load data
# Load data
(x_train_raw, y_train), (x_test_raw, y_test) = tf.keras.datasets.fashion_mnist.load_data()
x_train_raw = x_train_raw[..., None]
x_test_raw = x_test_raw[..., None]
# GAN data [-1, 1]
x_train_gan = (x_train_raw.astype("float32") - 127.5) / 127.5
# Classifier data [0, 1]
x_train_cls = x_train_raw.astype("float32") / 255.0
x_test_cls = x_test_raw.astype("float32") / 255.0
print("GAN training data:", x_train_gan.shape)
print("Classifier training data:", x_train_cls.shape)
print("Test data:", x_test_cls.shape)
```

```
# ---------------- Generator ----------------
G = tf.keras.Sequential([
    layers.Input(shape=(100,)),
    layers.Dense(7*7*256, use_bias=False),
    layers.BatchNormalization(),
    layers.LeakyReLU(),
    layers.Reshape((7,7,256)),
    layers.Conv2DTranspose(128, 4, 2, padding="same", use_bias=False),
    layers.BatchNormalization(),
    layers.LeakyReLU(),
    layers.Conv2DTranspose(64, 4, 2, padding="same", use_bias=False),
    layers.BatchNormalization(),
    layers.LeakyReLU(),
    layers.Conv2D(1, 3, padding="same", activation="tanh")
])
# ---------------- Discriminator ----------------
D = tf.keras.Sequential([
    layers.InputLayer(input_shape=(28,28,1)),
    layers.Conv2D(64, 4, 2, padding="same"),
    layers.LeakyReLU(0.2),
    layers.Dropout(0.3),
    layers.Conv2D(128, 4, 2, padding="same"),
    layers.LeakyReLU(0.2),
    layers.Dropout(0.3),
    layers.Flatten(),
    layers.Dense(1, activation="sigmoid")
])
D.compile(optimizer=tf.keras.optimizers.Adam(0.0002, 0.5), loss="binary_crossentropy")
# GAN
D.trainable = False
GAN = tf.keras.Sequential([G, D])
GAN.compile(optimizer=tf.keras.optimizers.Adam(0.0002, 0.5), loss="binary_crossentropy")
# Train GAN
```

```
epochs = 50
batch_size = 128
for epoch in range(epochs):
    for i in range(0, len(x_train_gan), batch_size):
        real = x_train_gan[i:i+batch_size]
        n = len(real)

        noise = np.random.normal(0, 1, (n, 100))
        fake = G.predict(noise, verbose=0)
        D.trainable = True
        D.train_on_batch(real, np.ones((n,1))*0.9)
        D.train_on_batch(fake, np.zeros((n,1)))
        D.trainable = False
        noise = np.random.normal(0, 1, (n,100))
        GAN.train_on_batch(noise, np.ones((n,1)))
    print("Epoch:", epoch+1)
# Generate images
noise = np.random.normal(0, 1, (16,100))
generated = G.predict(noise, verbose=0)
generated = (generated + 1) / 2
# Display generated images
plt.figure(figsize=(8,8))
for i in range(16):
    plt.subplot(4,4,i+1)
    plt.imshow(generated[i].squeeze(), cmap="gray")
    plt.axis("off")
plt.suptitle("GAN Generated Images")
plt.tight_layout()
plt.show()
# Generate synthetic data
num_fake = 5000
noise = np.random.normal(0, 1, (num_fake,100))
fake_images = G.predict(noise, verbose=0)
```

```
fake_images = (fake_images + 1) / 2
print("Generated images:", fake_images.shape)
# Classifier
def create_classifier():
    model = tf.keras.Sequential([
        layers.Input(shape=(28,28,1)),
        layers.Conv2D(32,3,activation="relu"),
        layers.MaxPooling2D(),
        layers.Conv2D(64,3,activation="relu"),
        layers.MaxPooling2D(),
        layers.Flatten(),
        layers.Dense(64,activation="relu"),
        layers.Dense(10,activation="softmax")
    ])
    model.compile(optimizer="adam", loss="sparse_categorical_crossentropy", metrics=["accuracy"])
    return model
# Original classifier
print("Training original classifier...")
model_original = create_classifier()
model_original.fit(x_train_cls, y_train, epochs=3, batch_size=128, verbose=1)
_, accuracy_original = model_original.evaluate(x_test_cls, y_test, verbose=0)
# Augmented dataset
fake_labels = np.random.choice(y_train, num_fake)
x_augmented = np.concatenate([x_train_cls, fake_images])
y_augmented = np.concatenate([y_train, fake_labels])
idx = np.random.permutation(len(x_augmented))
x_augmented = x_augmented[idx]
y_augmented = y_augmented[idx]
print("Augmented data:", x_augmented.shape)
# Augmented classifier
print("Training augmented classifier...")
model_augmented = create_classifier()
model_augmented.fit(x_augmented, y_augmented, epochs=3, batch_size=128, verbose=1)
```

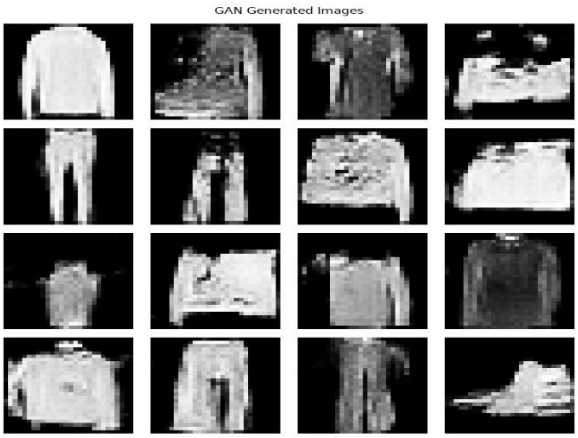
```
_, accuracy_augmented = model_augmented.evaluate(x_test_cls, y_test, verbose=0)
# Results
print("\nOriginal Accuracy:", accuracy_original*100, "%")
print("GAN Augmented Accuracy:", accuracy_augmented*100, "%")
print("Change:", (accuracy_augmented-accuracy_original)*100, "%")

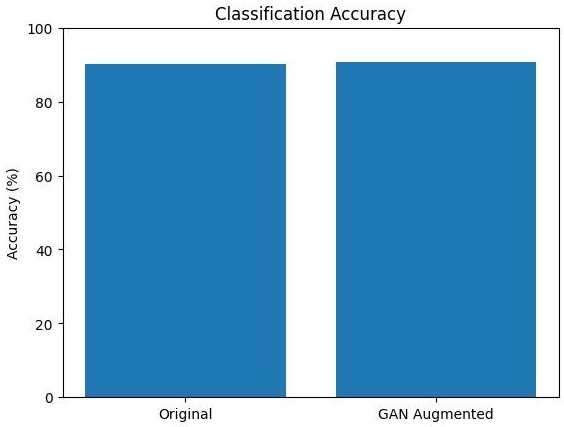
# Accuracy graph
plt.figure(figsize=(7,5))
plt.bar(["Original", "GAN Augmented"], [accuracy_original*100, accuracy_augmented*100])
plt.ylabel("Accuracy (%)")
plt.title("Classification Accuracy")
plt.ylim(0,100)
plt.show()
```

OUTPUT

```
GAN training data: (60000, 28, 28, 1)
Classifier training data: (60000, 28, 28, 1)
Test data: (10000, 28, 28, 1)

Original Accuracy: 90.20000100135803 %
GAN Augmented Accuracy: 90.6000018119812 %
Change: 0.40000081062316895 %
```





RESULT

The Generative Adversarial Network (GAN) was successfully implemented using the Fashion-MNIST dataset to generate synthetic images. The generated images were combined with the original training data to create an augmented dataset. A CNN classifier was trained using both the original and augmented datasets, and their classification accuracies were compared. The results showed that GAN-based data augmentation can improve the classification performance of the CNN model by providing additional training samples.

EX.NO: 10 DATE:	**Build a recommendation system using deep learning from a sales dataset (e.g., transactional data from an e-commerce platform). Implement collaborative filtering or neural matrix factorization to suggest products to users.**

AIM

To develop a deep learning-based product recommendation system using neural collaborative filtering, in which user and product embeddings are learned from e-commerce transaction data, and to use the trained model to suggest the top-N products a customer has not yet purchased.

ALGORITHM

1. Import the required NumPy, Pandas, TensorFlow/Keras and Scikit-learn libraries.
2. Read the sales dataset from the CSV file and remove rows with missing customer ID, product ID, product name, product category, quantity or order status.
3. Convert the quantity column to numeric, standardize the order status text, and keep only completed orders with a quantity greater than zero.
4. Group the data by customer and product, summing the quantity, to create positive customer-product interactions with label 1.
5. Map each customer ID and product ID to a unique integer index for use in the embedding layers.
6. Generate negative samples by randomly selecting up to four products that each customer has not purchased and assigning them label 0.
7. Combine the positive and negative samples and split them into training and validation sets (80:20) using stratified sampling.
8. Build the neural collaborative filtering model using separate Embedding layers for users and items, concatenate the two embedding vectors, and pass them through Dense (ReLU), Dropout and Sigmoid output layers.
9. Compile the model using the Adam optimizer, binary cross-entropy loss and AUC metric, and train it for 30 epochs with a batch size of 64.
10. Create a recommendation function that predicts scores for all products not yet purchased by a customer, ranks them in descending order, and displays the top-N products with their names, categories and scores.

SOFTWARE REQUIREMENTS

- **Programming Language:** Python 3.x
- **Deep Learning Framework:** TensorFlow / Keras
- **Libraries:** NumPy, Pandas, Scikit-learn, TensorFlow, Keras
- **Dataset:** E-commerce sales transaction data (CSV file with customer_id, product_id, product_name, product_category, quantity and order_status columns)

- **IDE/Platform:** Jupyter Notebook, Google Colab, VS Code, or PyCharm
- **Required Packages:** tensorflow, keras, numpy, pandas, scikit-learn

PROGRAM

```
import numpy as np
import pandas as pd
import tensorflow as tf
from keras import layers, Model
from sklearn.model_selection import train_test_split
# 1. Load and preprocess data
df = pd.read_csv(r"D:\Projects\Learning\DL_and_ML_tryouts_supplements\Experiment-10.csv")
df = df.dropna(subset=["customer_id", "product_id", "product_name",
                       "product_category", "quantity", "order_status"])
df["quantity"] = pd.to_numeric(df["quantity"], errors="coerce")
df["order_status"] = df["order_status"].astype(str).str.strip().str.lower()
df = df[(df["order_status"] == "completed") & (df["quantity"] > 0)].copy()
# 2. Create customer-product interactions
interactions = df.groupby(["customer_id", "product_id"], as_index=False).agg(
    quantity=("quantity", "sum"))
interactions["label"] = 1.0
customers = interactions["customer_id"].unique()
products = df.drop_duplicates("product_id").set_index("product_id")
user_map = {u: i for i, u in enumerate(customers)}
item_map = {p: i for i, p in enumerate(interactions["product_id"].unique())}
idx_to_item = {i: p for p, i in item_map.items()}
interactions["user_idx"] = interactions["customer_id"].map(user_map)
interactions["item_idx"] = interactions["product_id"].map(item_map)
num_users, num_items = len(user_map), len(item_map)
# 3. Generate negative samples
rng = np.random.default_rng(42)
purchased = interactions.groupby("user_idx")["item_idx"].apply(set).to_dict()
negative = [(u, i, 0.0) for u, items in purchased.items()
            for i in rng.choice([i for i in range(num_items) if i not in items],
                                size=min(4, len([i for i in range(num_items) if i not in items])),
                                replace=False)]
data = pd.concat([
    interactions[["user_idx", "item_idx", "label"]],
```

```
        pd.DataFrame(negative, columns=["user_idx", "item_idx", "label"])
], ignore_index=True)
# 4. Split data
train, val = train_test_split(data, test_size=0.2, random_state=42,
                                        stratify=data["label"])
def make_inputs(d):
        return {"user_input": d["user_idx"].to_numpy(dtype=np.int32),
                "item_input": d["item_idx"].to_numpy(dtype=np.int32)}
X_train, y_train = make_inputs(train), train["label"].to_numpy(dtype=np.float32)
X_val, y_val = make_inputs(val), val["label"].to_numpy(dtype=np.float32)
# 5. Build recommendation model
user_input = layers.Input(shape=(1,), dtype="int32", name="user_input")
item_input = layers.Input(shape=(1,), dtype="int32", name="item_input")
u = layers.Flatten()(layers.Embedding(num_users, 32)(user_input))
i = layers.Flatten()(layers.Embedding(num_items, 32)(item_input))
x = layers.Concatenate()([u, i])
x = layers.Dense(64, activation="relu")(x)
x = layers.Dropout(0.2)(x)
x = layers.Dense(32, activation="relu")(x)
output = layers.Dense(1, activation="sigmoid")(x)
model = Model([user_input, item_input], output)
model.compile(optimizer="adam", loss="binary_crossentropy",
                    metrics=[tf.keras.metrics.AUC(name="auc")])
# 6. Train model
model.fit(X_train, y_train, validation_data=(X_val, y_val),
              epochs=30, batch_size=64)
# 7. Recommend products
def recommend_products(customer_id, top_n=5):
        u = user_map[customer_id]
        candidates = [i for i in range(num_items)
                            if i not in purchased.get(u, set())]
        if not candidates:
              return pd.DataFrame()
        scores = model.predict({
              "user_input": np.full(len(candidates), u, dtype=np.int32),
              "item_input": np.array(candidates, dtype=np.int32)
```

```
    }, verbose=0).flatten()
    ranked = np.argsort(scores)[::-1][:top_n]
    return pd.DataFrame([{
        "product_id": idx_to_item[candidates[j]],
        "product_name": products.loc[idx_to_item[candidates[j]], "product_name"],
        "product_category": products.loc[idx_to_item[candidates[j]], "product_category"],
        "recommendation_score": round(float(scores[j]), 4)
    } for j in ranked])
# 8. Display recommendations
customer = customers[10]
print("Customer:", customer)
print(recommend_products(customer, 5).to_string(index=False))
```

OUTPUT

Epoch 1/30 — auc: 0.4976, loss: 0.6683, val_auc: 0.4972, val_loss: 0.6677

Epoch 30/30 — auc: 0.9995, loss: 0.0221, val_auc: 0.5089, val_loss: 3.5972

Sample Recommendations:

Customer: CUST-000011

product_id	product_name	product_category	recommendation_score
PROD-0056	Tennis Racket	Sports & Outdoors	1.0000
PROD-0075	Breakfast Cereal	Grocery	1.0000
PROD-0071	Coffee Beans	Grocery	0.9999
PROD-0048	Reading Lamp	Books & Stationery	0.9998
PROD-0051	Yoga Mat	Sports & Outdoors	0.9987

RESULT

The neural network model was trained using customer and product embeddings to predict purchase preferences. It generates recommendation scores for unpurchased products, ranks them in descending order, and displays the top 5 recommended products with their IDs, names, categories, and scores.

EX.NO: 11 DATE:	**Implement an image captioning system that generates textual descriptions of images using a combination of Convolutional Neural Networks (CNNs) for feature extraction and Long Short-Term Memory (LSTM) networks for sequence generation. Train the model on a dataset like MS COCO or Flickr 8k and evaluate it using BLEU or METEOR scores.**

AIM

To develop an image captioning system using deep learning techniques. To extract image features using InceptionV3 and generate meaningful captions using an LSTM network.

ALGORITHM

1. Import the required Python libraries and deep learning modules.
2. Load the Flickr8k image dataset and caption text files.
3. Count the total images, captions, and training, validation, and testing images.
4. Remove image filenames that do not exist in the dataset directory.
5. Load captions, convert text to lowercase, and remove punctuation.
6. Add start and end sequence tokens to each caption.
7. Load the pretrained InceptionV3 model with ImageNet weights.
8. Extract image features and store them for further processing.
9. Tokenize the training captions and determine the vocabulary size and maximum caption length.
10. Convert captions into input sequences and corresponding target words.
11. Build an image captioning model using Dense, Embedding, Dropout, and LSTM layers.
12. Compile and train the model using the Adam optimizer and categorical sequence prediction loss.
13. Save the trained model, plot training and validation loss, and generate a caption for a test image..

SOFTWARE REQUIREMENTS

- **Programming Language:** Python
- **Libraries:** TensorFlow, Keras, NumPy, Matplotlib, Pillow
- **Dataset:** Flickr8k Image Captioning Dataset
- **IDE:** Jupyter Notebook / Google Colab / VS Code
- **Hardware:** Computer with sufficient RAM and storage

PROGRAM

```
import os

import string

import pickle

import random

import numpy as np

import tensorflow as tf

import matplotlib.pyplot as plt

from PIL import Image

from tensorflow.keras.applications import InceptionV3

from tensorflow.keras.applications.inception_v3 import preprocess_input

from tensorflow.keras.preprocessing.sequence import pad_sequences

from tensorflow.keras.preprocessing.text import Tokenizer

from tensorflow.keras.models import Model

from tensorflow.keras.layers import Input, Dense, LSTM, Embedding, Dropout, add

# STEP 1: DATASET PATHS

image_dir = r"D:/Work/Sairam/Subjects Handled/5. June-Dec 2026/Deep Learning
Lab/Exp_11/Data/Flickr8k_Dataset/Flicker8k_Dataset"

text_dir = r"D:/Work/Sairam/Subjects Handled/5. June-Dec 2026/Deep Learning
Lab/Exp_11/Data/Flickr8k_text"

caption_file = os.path.join(text_dir, "Flickr8k.token.txt")

if not os.path.exists(caption_file):

    caption_file = os.path.join(text_dir, "Flickr8k.token")

train_file = os.path.join(text_dir, "Flickr_8k.trainImages.txt")

val_file = os.path.join(text_dir, "Flickr_8k.devImages.txt")

test_file = os.path.join(text_dir, "Flickr_8k.testImages.txt")

# STEP 2: DATASET SUMMARY

image_count = sum(

    f.lower().endswith((".jpg", ".jpeg", ".png"))

  for f in os.listdir(image_dir)
```

```
)
with open(caption_file, "r", encoding="utf-8") as f:
    caption_count = sum(1 for line in f if line.strip())
def load_split(filename):
    with open(filename, "r", encoding="utf-8") as f:
        return [line.strip() for line in f if line.strip()]
train_names = load_split(train_file)
val_names = load_split(val_file)
test_names = load_split(test_file)
print("========== Flickr8k Dataset Summary ==========")
print("Total images:", image_count)
print("Total captions:", caption_count)
print("Training images:", len(train_names))
print("Validation images:", len(val_names))
print("Testing images:", len(test_names))
# Remove filenames whose image files do not exist
train_names = [
    n for n in train_names
    if os.path.isfile(os.path.join(image_dir, n))
]
val_names = [
    n for n in val_names
    if os.path.isfile(os.path.join(image_dir, n))
]
test_names = [
    n for n in test_names
    if os.path.isfile(os.path.join(image_dir, n))
]
# STEP 3: LOAD AND CLEAN CAPTIONS
```

```
def load_captions(filename):
    captions = {}
    with open(filename, "r", encoding="utf-8") as f:
        for line in f:
            parts = line.strip().split("\t")
            if len(parts) != 2:
                continue
            name = parts[0].split("#")[0]
            caption = parts[1].lower()
            caption = caption.translate(
                str.maketrans("", "", string.punctuation)
            )
            caption = "startseq " + caption + " endseq"
            captions.setdefault(name, []).append(caption)
    return captions
captions = load_captions(caption_file)
print("\nCaptions loaded:", len(captions))
# STEP 4: SELECT ALL IMAGES FROM THE SPLITS
all_names = list(dict.fromkeys(train_names + val_names + test_names))
# STEP 5: EXTRACT IMAGE FEATURES USING INCEPTIONV3
base_model = InceptionV3(weights="imagenet")
feature_model = Model(
    inputs=base_model.input,
    outputs=base_model.layers[-2].output
)
features = {}
for i, name in enumerate(all_names):
    image_path = os.path.join(image_dir, name)
    image = Image.open(image_path).convert("RGB")
```

```
    image = image.resize((299, 299))
    image = np.array(image, dtype=np.float32)
    image = np.expand_dims(image, axis=0)
    image = preprocess_input(image)
    feature = feature_model.predict(image, verbose=0)
    features[name] = feature[0]
    if (i + 1) % 500 == 0:
        print("Features extracted:", i + 1, "/", len(all_names))
np.save("D:/Work/Sairam/Subjects Handled/5. June-Dec 2026/Deep Learning
Lab/Exp_11/flickr8k_features.npy", features)
print("Image feature extraction completed.")
# STEP 6: PREPARE CAPTIONS AND TOKENIZER
train_captions = []
for name in train_names:
    train_captions.extend(captions[name])
tokenizer = Tokenizer()
tokenizer.fit_on_texts(train_captions)
vocab_size = len(tokenizer.word_index) + 1
max_length = max(len(c.split()) for c in train_captions)
print("Vocabulary size:", vocab_size)
print("Maximum caption length:", max_length)
with open("flickr8k_tokenizer.pkl", "wb") as f:
    pickle.dump(tokenizer, f)
# STEP 7: CREATE TRAINING SEQUENCES
def create_sequences(image_names):
    X_image = []
    X_sequence = []
    y_word = []
    for name in image_names:
```

```
        for caption in captions[name]:
            sequence = tokenizer.texts_to_sequences([caption])[0]
            for i in range(1, len(sequence)):
                input_seq = sequence[:i]
                output_word = sequence[i]
                input_seq = pad_sequences(
                    [input_seq], maxlen=max_length
                )[0]
                X_image.append(features[name])
                X_sequence.append(input_seq)
                y_word.append(output_word)
    return (
        np.array(X_image, dtype=np.float32),
        np.array(X_sequence, dtype=np.int32),
        np.array(y_word, dtype=np.int32)
    )
X1_train, X2_train, y_train = create_sequences(train_names)
X1_val, X2_val, y_val = create_sequences(val_names)
print("Training sequences:", len(y_train))
print("Validation sequences:", len(y_val))
# STEP 8: BUILD THE IMAGE CAPTIONING MODEL
image_input = Input(shape=(2048,))
image_features = Dropout(0.3)(image_input)
image_features = Dense(256, activation="relu")(image_features)
caption_input = Input(shape=(max_length,))
caption_features = Embedding(
    vocab_size, 256, mask_zero=True
)(caption_input)
caption_features = Dropout(0.3)(caption_features)
```

```
caption_features = LSTM(256)(caption_features)
decoder = add([image_features, caption_features])
decoder = Dense(256, activation="relu")(decoder)
output = Dense(vocab_size, activation="softmax")(decoder)
model = Model(
    inputs=[image_input, caption_input],
    outputs=output
)
model.compile(
    loss="sparse_categorical_crossentropy",
    optimizer="adam"
)
model.summary()
# STEP 9: TRAIN THE MODEL
history = model.fit(
    [X1_train, X2_train],
    y_train,
    validation_data=([X1_val, X2_val], y_val),
    epochs=5,
    batch_size=256
)
model.save("flickr8k_caption_model.keras")
print("Model training completed and saved.")
# STEP 10: PLOT TRAINING AND VALIDATION LOSS
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Image Captioning Model Loss")
```

```
plt.legend()
plt.show()
# STEP 11: GENERATE A CAPTION FOR AN IMAGE
def generate_caption(name):
    image_feature = features[name].reshape(1, 2048)
    caption = "startseq"
    for i in range(max_length):
        sequence = tokenizer.texts_to_sequences([caption])[0]
        sequence = pad_sequences(
            [sequence], maxlen=max_length
        )
        prediction = model.predict(
            [image_feature, sequence], verbose=0
        )
        word_index = np.argmax(prediction)
        word = tokenizer.index_word.get(word_index)
        if word is None or word == "endseq":
            break
        caption += " " + word
    return caption.replace("startseq", "").strip()
# STEP 12: DISPLAY A TEST IMAGE AND ITS CAPTION
test_image = test_names[0]
image = Image.open(os.path.join(image_dir, test_image))
plt.imshow(image)
plt.axis("off")
plt.title(generate_caption(test_image))
plt.show()
```

OUTPUT

========== Flickr8k Dataset Summary ==========

Total images: 8091

Total captions: 40460

Training images: 6000

Validation images: 1000

Testing images: 1000

Captions loaded: 8092

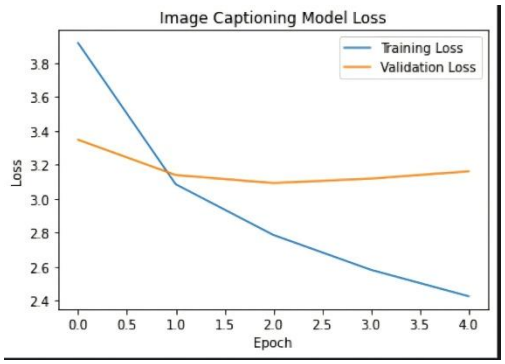
Vocabulary size: 7633

Maximum caption length: 38

Training sequences: 353639

Validation sequences: 58781





RESULT

The neural network model was trained using customer and product embeddings to predict purchase preferences. It generates recommendation scores for unpurchased products, ranks them in descending order, and displays the top 5 recommended products with their IDs, names, categories, and score.